

CRISTOFF STAN

Senior Software Engineer

Location: Tallinn, Harjumaa, Estonia | Email: cristoffstan@gmail.com
LinkedIn: <https://www.linkedin.com/in/cristoff-stan-b53490362/> | Phone Number: +372 58619311

PROFESSIONAL SUMMARY

Detail-oriented Senior Software Engineer with 9+ years of experience delivering robust backend systems and refactoring complex codebases across distributed environments. Specialized in writing and evaluating production-grade code with a critical focus on Rust, code correctness, and performance optimization. Proven ability to assess logic, identify edge cases, enforce security best practices, and articulate reasoning in asynchronous communication. Passionate about clean code, language specifications, and guiding LLMs through expert feedback loops in reinforcement learning setups.

TECHNICAL SKILLS

- **Languages:** Rust, TypeScript, JavaScript, Python, C++, Go, SQL, Shell Scripting
- **Tools:** Git, Docker, Postman, VS Code, JIRA, GitHub, GitLab
- **Code Review & Refactoring:** Logic error detection, performance profiling, secure coding patterns, idiomatic Rust practices
- **Frameworks & Libraries:** Actix, Tokio, Serde, FastAPI, Node.js, React, Express.js
- **Testing & QA:** Unit testing, integration testing, test-driven development, coverage analysis
- **CI/CD & DevOps:** GitHub Actions, Jenkins, Docker Compose
- **Databases:** PostgreSQL, Redis, MongoDB
- **Other:** LLM fine-tuning feedback cycles, reinforcement learning fundamentals, language specification compliance

WORK EXPERIENCE

Senior Software Engineer

Sensidev

September 2021 – December 2024

Timișoara, Romania (Remote)

- Refactored and optimized critical Rust-based services for cross-platform deployments, reducing latency by 30% and improving code readability and maintainability.
- Led internal code audits and implemented standardized review criteria focused on performance, idiomatic structure, and security compliance across Rust and TypeScript codebases.
- Collaborated with research teams to evaluate and correct AI-generated code snippets by integrating contextual logic, data validation, and system design expectations.
- Authored detailed feedback documentation on multi-solution evaluations, explaining trade-offs between readability, performance, and system compatibility.

- Introduced regression prevention workflows by designing custom test suites that validate AI-edited code, contributing directly to RLHF reward signal generation.
- Rewrote several performance-critical Rust modules with async-safe implementations using Tokio and Actix, eliminating thread-blocking I/O and race conditions.
- Contributed directly to the training loop of LLMs by editing, annotating, and ranking AI-generated solutions with verbose, instructive justifications and error analysis.
- Promoted async-first communication across global team channels, contributing to seamless handoffs and reviews without reliance on real-time standups or meetings.

Software Engineer

Syscom Digital

October 2019 – August 2021

Bucharest, Romania (Remote)

- Delivered backend APIs and CLI tooling in Rust and Python for campaign automation, focusing on code reusability, concurrency safety, and deterministic outputs.
- Reviewed and corrected machine-generated modules during internal research projects, offering granular reasoning for code design, logic gaps, and alternative structures.
- Refined internal linters and formatters to enforce consistent code conventions across teams and conducted weekly knowledge-sharing reviews on language-specific best practices.
- Spearheaded multiple code improvement initiatives, identifying and fixing inefficient patterns in legacy Rust systems while maintaining full backward compatibility.
- Built sandbox environments for code experimentation and AI-generated test case validation, simulating edge conditions and hard-to-reproduce bug patterns.
- Conducted static code analysis and integrated automated feedback loops for pre-merge evaluations with contextual annotations and scoring on logic and style adherence.
- Served as code quality liaison between the engineering and research teams, clarifying complex performance implications and enabling effective model feedback.
- Mentored junior developers on idiomatic Rust usage, data ownership models, borrow checker nuances, and leveraging type systems for compile-time safety.

Software Developer

Soft Build

June 2016 – July 2019

Timișoara, Romania (Remote)

- Developed and maintained distributed systems in Rust and C++ for data aggregation and transformation, focusing on safety and performance.
- Conducted in-depth code reviews of junior team submissions, offering structured critique on performance, memory handling, modularity, and naming conventions.
- Engineered internal tools for automatic feedback scoring of multiple solution paths to programming tasks, which later served as foundations for AI training experiments.
- Identified anti-patterns and refactored monolithic modules into testable, composable functions with deterministic behaviors under constrained runtime budgets.
- Implemented diagnostics pipelines to analyze AI-generated snippets for correctness, variable scoping issues, and resource handling bugs.
- Supported documentation of code evaluations and model comparisons by authoring structured markdown templates for feedback capture and traceability.
- Designed fallback mechanisms for fault-tolerant operations, simulating failure conditions and ensuring stability through rollback paths and retry loops.
- Participated in low-level system diagnostics and debugging, using Rust's unsafe code blocks responsibly to optimize hot paths with zero-cost abstractions.

Software Developer
Decima Digital
November 2015 – April 2016
Tallinn, Estonia

- Wrote core backend logic in Rust for a supply chain analytics platform, balancing memory safety, concurrency, and business rule enforcement.
- Collaborated closely with QA to repair broken AI-generated test scenarios, mapping failures to logical code gaps and proposing idiomatic rewrites.
- Refactored spaghetti logic into declarative pipelines using iterator combinators and functional patterns, significantly reducing code complexity.
- Optimized multithreaded data parsers for real-time analytics ingestion using Rust's ownership model and lock-free design.
- Documented rationales for technical decisions and performance choices in internal code reviews, enabling model training from engineering behaviors.
- Reviewed early stage AI-generated utility code and provided feedback on maintainability, runtime efficiency, and idiomatic structure.
- Integrated benchmark harnesses to profile system behavior under simulated AI-generated misuse conditions.
- Proposed and implemented fine-grained modularization for testability and incremental compilation.

Software Developer Intern
Borna Technology
September 2014 – August 2015
Tallinn, Estonia (On-Site)

- Contributed to backend systems in C++ and Rust, with special focus on syntax compliance, error handling strategies, and minimal runtime overhead.
- Compared multiple code solutions for standard problems, noting down clarity, maintainability, edge case handling, and stylistic choices.
- Rewrote faulty logic suggested by code generation models during exploratory trials, fixing silent failures and infinite loops through targeted rewrites.
- Developed a habit of consulting compiler internals and language specs to understand nuanced behavior of type coercions, trait resolution, and lifetimes.
- Built and maintained test cases for comparison of functionally equivalent implementations with varying performance characteristics.
- Took initiative to highlight unreachable or dead code segments in peer-written modules, proposing simplifications based on control flow analysis.
- Maintained detailed write-ups during review sessions, often dissecting five or more alternative code approaches and reasoning trade-offs in prose.
- Enthusiastically adopted a feedback-driven improvement cycle for both manual and AI-authored code, forming early foundations of RLHF-style workflows.

EDUCATION

Tallinn University

Bachelor's Degree in Computer Science,
2010 – 2014

CERTIFICATIONS

- ❖ Certified Rust Developer – Coursera (Issued 2024)
 - ❖ Secure Coding Practices – Rust Edition (OWASP Rust Training, 2023)
 - ❖ Advanced Code Review Masterclass – Udemy (2022)
 - ❖ Asynchronous Communication & Remote Collaboration – RemoteHow (2022)
-

PROJECTS

- ✓ **Rust Refactoring Engine for AI Feedback Training:** Designed a tool to analyze, refactor, and annotate AI-generated Rust code with detailed line-level feedback for training LLMs. Focused on performance profiling, lint coverage, and test-driven scoring metrics.
- ✓ **RLHF Feedback Suite:** Built a ranking and critique interface for human-in-the-loop feedback on generated code snippets, supporting side-by-side comparisons and inline correction with justifications.
- ✓ **Code Critique Corpus:** Assembled a dataset of annotated reviews across real and AI-generated Rust submissions, detailing reasons behind code quality scores for model reinforcement.